Migration Guide for Remaining Tools

Overview

This guide explains how to migrate the remaining tools from `main.py` to the modular structure.

File Structure

```
your_project/
├── main.py                 # Main application (refactored)
├── tool_loader.py          # Tool loading utility
├── tools/                  # All tools go here
│   ├── base_tool.py        # Base class for all tools
│   ├── installers.py       # ✓ Completed
│   ├── folder_creator.py   # ✓ Completed
│   ├── batch_renamer.py    # ✓ Completed
│   ├── playback.py         # ✓ Completed
│   ├── timecode_calculator.py # ✓ Completed
│   ├── subtitle_to_markers.py # ✓ Completed
│   ├── storage_monitor.py # ✓ Completed
│   ├── consolidate_media.py  # TODO
│   ├── convert_edls.py       # TODO
│   ├── guides.py             # TODO
│   ├── quality_control.py    # TODO
│   └── metadata_viewer.py    # TODO
├── foldertemplates/        # Template CSV files
└── (other existing files like batchrenamer.py, media_browser.py, etc.)
```

How to Create a New Tool Module

1. Basic Structure

Every tool must have a `Tool` class that inherits from `BaseTool`:

python

```python
import customtkinter as ctk
from base_tool import BaseTool

class Tool(BaseTool):
    def __init__(self):
        super().__init__()
        self.name = "Tool Name"  # Appears in menu
        self.description = "Tool description"

    def create_window(self, parent):
        window = self.create_tool_window(parent, "Window Title", width, height)

        # Add UI elements

        return window
```

2. Migration Steps

Step 1: Create the tool file

Create a new `.py` file in the `tools/` directory with the tool name (e.g., `consolidate_media.py`)

Step 2: Copy the relevant code

From `main.py`, copy:

- The content method (e.g., `consolidate_media_content`)
- Any helper methods used by that tool
- Import statements needed

Step 3: Adapt the code

- Replace `self` references that refer to the main app with local references
- Store UI elements as instance variables if they need to be accessed by methods
- Add `self.window = window` to store window reference if needed

Step 4: Handle imports

- Import only what's needed for that specific tool
- Use relative imports for shared modules

3. Example Migration: Consolidate Media

Here's how to migrate the Consolidate Media tool:

python

```python
import customtkinter as ctk
import tkinter as tk
from tkinter import filedialog, messagebox
import os
import csv
import shutil
from base_tool import BaseTool

class Tool(BaseTool):
    def __init__(self):
        super().__init__()
        self.name = "Consolidate Media"
        self.description = "Collect media files based on CSV lists"

    def create_window(self, parent):
        window = self.create_tool_window(parent, "Consolidate Media", 900, 700)

        # Copy the UI creation code from consolidate_media_content
        # Replace frame parameter with window
        # Store UI elements as self.element_name

        # Store window reference
        self.window = window

        return window

    # Copy helper methods like browse_file, run_consolidation, etc.
    # Make sure to update any references
```

4. Complex Tools with External Dependencies

For tools that use external widgets (like BatchRenamer, MediaBrowser):

- Import the external widget
- Create and configure it in the window
- The tool acts as a wrapper

5. Testing

After creating each tool:

1. Run `main.py`
2. Click the tool in the menu
3. Verify the window opens correctly
4. Test all functionality
5. Check that closing and reopening works

Benefits of This Structure

1. **Modularity**: Each tool is independent
2. **Maintainability**: Easy to update individual tools
3. **Extensibility**: Simple to add new tools
4. **Organization**: Clear file structure
5. **Reusability**: Tools can be used in other projects
6. **Window Management**: Each tool runs in its own window

Notes

- The main window stays open and acts as a launcher
- Each tool opens in a separate window
- Windows can be closed and reopened without affecting others
- The `tool_loader.py` automatically discovers new tools in the `tools/` directory
- Tool names in the menu are determined by the `self.name` property